`Phyloom`: a flexible and extensible framework for simulating phylogenetic trees and networks

Gabriele Marino[1,2], Daniele Silvestro[1,2], and Tanja Stadler[1,2]

[1]Department of Biosystems Science and Engineering, ETH Zurich, Basel, Switzerland
[2]SIB Swiss Institute of Bioinformatics, Basel, Switzerland

Abstract

Summary: Simulated phylogenies underpin the validation of phylogenetic and phylodynamic inference methods, simulation-based hypothesis testing, and the training of likelihood-free estimators. Existing simulators cover many established models collectively, but each offers only a fixed menu of features on its own. Implementing even a slight tweak to a model—a common necessity when developing new methods—often forces users to write a simulator from scratch. We present `phyloom`, a Python framework in which simulators are instead assembled from interchangeable parts: event rules on lineage primitives, generic time- and model-dependent rate functions, scheduled actions, and user-defined stopping and acceptance criteria. Configured entirely through YAML without code, `phyloom` unifies standard phylogenetic and phylodynamic model families within a single tool. Priors, rejection sampling, and parallel execution turn dataset generation into a single command. Plugins can add custom models—including non-Markovian dynamics with individual-level memory—in just a few dozen to a few hundred lines of code.

Availability and implementation: `phyloom` is implemented in Python (≥3.10), released under the MIT license, and available from PyPI (`pip install phyloom`) and from the GitHub repository https://github.com/gabriele-marino/phyloom. Documentation and tutorials are at https://gabriele-marino.github.io/phyloom/home.

Contact: gmarino@ethz.ch

1 Introduction

Phylogenetic trees and networks hold the evolutionary trace of how species, populations, or pathogen lineages have diversified. Phylodynamic methods use these phylogenies to quantify the processes that created them, including transmission, recovery, diversification, and population size changes over time [1]. Simulating phylogenies using explicit stochastic models serves as a core tool for developing phylodynamic models: it validates inference software through simulation-based calibration [2, 3], underpins posterior-predictive model assessment [4, 5] and identifiability studies [6], and, most recently, supplies the training data of likelihood-free estimators [7, 8].

A vast range of simulation tools exist to for a variety of different tree generating models: `TreeSim` [9] for birth–death models, `nosoi` [10] for epidemic transmission, `msprime` [11] for coalescent genealogies, and `SiPhyNetwork` [12] for birth–death–hybridization networks. Nonetheless, each has its own interface and configuration language, and a fixed menu of hardcoded models rather than reusable primitives. Combining features across tools, such as reticulation with a structured, time-varying epidemic, or adding a custom process thus means compromising on the model, editing a library's core code, or writing a new simulator, often from scratch. Even `ReMASTER` [13], a very flexible phylodynamic simulator, is restricted to reactions with mass-action propensities and piecewise-constant rates, i.e. to processes that obey a time-inhomogeneous chemical master equation: it cannot express a rate that depends on population size in any other functional form, or a lineage that carries individual memory.

Here we present `phyloom`, which builds simulations from small, interchangeable components—event rules, hazards, rate multipliers, and scheduled actions—rather than from a fixed catalog of models. Any event can have an arbitrary time-dependent hazard and a state-dependent rate multiplier, with custom stopping and acceptance criteria, so that birth–death processes, coalescent genealogies, and compartmental epidemics share a single framework. Custom logic, such as non-Markovian dynamics with per-lineage memory, is added through plugins without modifying the core simulator.

2 A generic event-driven engine

`Phyloom` separates what a stochastic event does from when it happens. A model holds a system state $X(t)$ (the current lineages, their discrete state labels, and any auxiliary quantities such as host pools) and a list of events. Each event pairs an event rule with a hazard. The rule counts the number c of opportunities to fire in the current state (for example, the number of current lineages in a given state, or the number of lineage pairs that can coalesce or hybridize) and, when selected, applies one opportunity drawn uniformly at random. The hazard $\lambda(t)$ gives the per-opportunity rate as a function of time, and an optional rate multiplier $m(X)$ scales it by any function of the current system state. `Phyloom` also supports scheduled actions, which are arbitrary operations executed at fixed times, such as sampling each lineage with a fixed probability, or introducing new lineages. Furthermore, actions and events can dynamically trigger or schedule additional actions and events during a running simulation, providing the flexibility to model complex scenarios—such as a runtime mutation event creating a new lineage class characterized by an increased transmission rate.

For every transition, the scheduler draws $E \sim \mathrm{Exp}(1)$ for each stochastic event and solves

$$c\, m(X) \int_{t_0}^{t} \lambda(s)\, \mathrm{d}s = E \tag{1}$$

for the proposed firing time t by inverting the cumulative hazard. The clock then advances to the earliest proposal, the earliest deterministic scheduled action, or a user-defined time limit, whichever comes first. Because the system state is constant between transitions, this is an exact stochastic simulation algorithm: with constant rates it reduces to the Next Reaction Algorithm [14, 15], and with time-varying hazards it simulates the corresponding non-homogeneous process exactly [16].

3 Flexible and extensible dataset generation

In `phyloom`, a configuration file specifies a model, a context of per-sample parameter priors, and stopping and acceptance rules (Fig. 1). For every sample, the configuration layer validates this specification and materializes the context into a concrete model, which the simulator core advances with the engine described above. A complete configuration is short: the file below generates 1000 trees of 100 sampled tips each under a constant-rate birth–death process, drawing the birth, death, and sampling rates anew from uniform priors for every tree:

```
output_dir: outputs/BD-minimal
n_samples: {train: 1000}
seed: 42
context:
  la: {type: uniform, low: 0.5, high: 1.5}
  mu: {type: uniform, low: 0.0, high: 0.4}
  psi: {type: uniform, low: 0.05, high: 0.3}
model:
  spec: bdsh
  init_state: I
  events:
    - rule: {spec: birth}
```

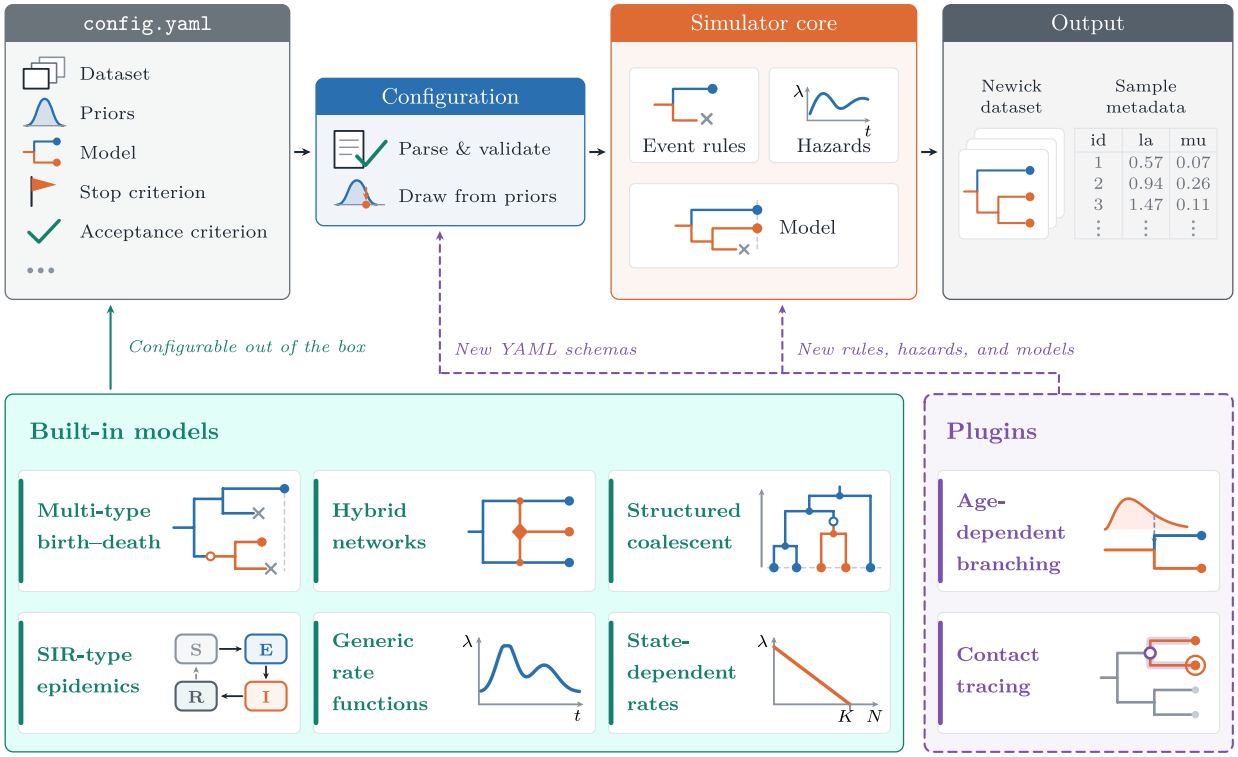


Figure 1: `Phyloom`'s pipeline, from configuration to dataset. A single YAML file defines how many simulations to run, the model as a list of event rules and their hazards, the priors on the model parameters, when a simulation trial ends and whether it is kept. The configuration layer validates it and materializes one model per simulation trial by drawing from the priors; the simulator core runs that model under a common scheduler. The dataset pairs each output phylogeny (annotated trees or networks in extended Newick format) with metadata records holding the drawn parameter values. `Phyloom` offers a library of built-in models, and users can add new ones as plugins.

```
      hazard: {spec: constant, rate: la}
    - rule: {spec: death}
      hazard: {spec: constant, rate: mu}
    - rule: {spec: sampling, removal: true}
      hazard: {spec: constant, rate: psi}
stop_criterion: model.count_sampled_lineages() == 100
acceptance_criterion: model.count_current_lineages() > 0
output_hooks:
  - model.history(model.get_sampled_lineages()).dump(output_dir / "tree.nwk")
```

Here `n_samples` specifies how many simulations to run for each split, i.e. partition of the dataset such as training, validation, or test (here only `train`). The runner writes each split to its own directory under `output_dir`, with one folder per simulation holding the files produced by the output hooks (here `train/data/1/tree.nwk`, `train/data/2/tree.nwk`, ...) and a single `train/metadata.csv`. Each `tree.nwk` is the simulated phylogeny in Newick format, with nodes labelled by identifier and lineage state (extended Newick for networks), and `metadata.csv` has one row per simulation with the drawn values of `la`, `mu`, and `psi`. Phylogenies and the parameters that generated them are thus paired by construction, as needed for simulation-based calibration and likelihood-free estimators [7].

`Phyloom` provides many standard phylogenetic models out of the box, with complete configurations available in the `examples` folder of the repository. These include constant-rate [17], skyline [18], multi-type [19], and state-dependent [20] birth–death models, with hybridization networks [12], structured [21] and serially sampled [22] coalescents [23] with arbitrary effective population-size

trajectories [24], and density-dependent compartmental epidemics [25]. In all of them, a hazard may be any expression of time, and any rate may be scaled by any function of the current state, such as clade diversity or prevalence.

`Phyloom` also supports optional post-commands, which execute after the output hooks for each sample. This allows external simulators of sequence or trait evolution to be integrated seamlessly into the pipeline. For example, simulating an alignment along every tree requires only a single post-command passing the tree to `AliSim` [26].

Processes outside the built-in families, such as non-Markovian dynamics with per-lineage memory, can be added by users as plugins, in a few dozen to a few hundred lines of code, without modifying `phyloom` itself; the `plugins` folder of the repository provides complete worked example projects, including contact tracing [27] and age-dependent branching [28] extensions.

4 Closing remarks

`Phyloom` builds a simulator from a composition of event rules, hazards, scheduled actions, and stopping and acceptance criteria, rather than offering a fixed model with tunable parameters or a menu spread across fragmented tools. The implementation is therefore very general: a single engine simulates phylogenetic trees and networks under birth–death, coalescent, and compartmental epidemic dynamics, with time-varying, density-dependent, and lineage-dependent rates. Command-line tools plug into the same pipeline without any code at all, so that sequence alignments can be seamlessly generated alongside the phylogenies.

Users can develop plugins to extend `phyloom` with their own models without writing a simulator from scratch, reusing its engine, configuration layer, and dataset pipeline at no extra cost. Planned extensions include approximate simulators for very large populations, where exact simulation becomes too slow.

Because of its generality, `phyloom` applies wherever phylogenies are used to study the processes that generated them, from macroevolution, epidemiology, and population genetics to single-cell and developmental lineage tracing, cancer evolution, and language evolution. By turning a new model into a configuration file or a small plugin, `phyloom` empowers researchers across these fields to investigate the underlying processes directly, streamlining the simulation pipeline so they can focus on the science rather than the implementation.

Computational performance

`Phyloom` generates large trees several times faster than a comparable pure-Python simulator, at the cost of higher memory use (benchmark details in the project repository), and supports parallel simulations across CPU cores with no additional setup.

Acknowledgments

We thank the Computational Evolution group at ETH Zurich for many insightful discussions, in particular Tim Vaughan for his guidance, Antoine Zwaans and Julia Pilarski on age-dependent branching, and Tobias Dieselhorst on time-dependent hazards. We also thank Anna Zhukova for discussions on contact tracing, and Alex Zarebski for encouraging us to release `phyloom`.

Funding

This work was funded by ETH Zurich.

Conflict of interest

None declared.

References

[1] Tanja Stadler, Carsten Magnus, Timothy G. Vaughan, Joëlle Barido-Sottani, Veronika Bošková, Jana S. Huisman, and Julija Pečerska. *Decoding Genomes: From Sequences to Phylodynamics*. ETH Zurich, Zurich, 2024. URL https://hdl.handle.net/20.500.11850/664449.

[2] Sean Talts, Michael Betancourt, Daniel Simpson, Aki Vehtari, and Andrew Gelman. Validating Bayesian inference algorithms with simulation-based calibration, 2018.

[3] Fábio K. Mendes, Remco Bouckaert, Luiz M. Carvalho, and Alexei J. Drummond. How to validate a Bayesian evolutionary model. *Systematic Biology*, 74(1):158–175, 2025. doi: 10.1093/sysbio/syae064.

[4] Jonathan P. Bollback. Bayesian model adequacy and choice in phylogenetics. *Molecular Biology and Evolution*, 19(7):1171–1180, 2002. doi: 10.1093/oxfordjournals.molbev.a004175.

[5] Sebastian Duchene, Remco Bouckaert, David A. Duchene, Tanja Stadler, and Alexei J. Drummond. Phylodynamic model adequacy using posterior predictive simulations. *Systematic Biology*, 68(2):358–364, 2019. doi: 10.1093/sysbio/syy048.

[6] Stilianos Louca, Angela McLaughlin, Ailene MacPherson, Jeffrey B. Joy, and Matthew W. Pennell. Fundamental identifiability limits in molecular epidemiology. *Molecular Biology and Evolution*, 38(9):4010–4024, 2021. doi: 10.1093/molbev/msab149.

[7] J. Voznica, A. Zhukova, V. Boskova, E. Saulnier, F. Lemoine, M. Moslonka-Lefebvre, and O. Gascuel. Deep learning from phylogenies to uncover the epidemiological dynamics of outbreaks. *Nature Communications*, 13:3896, 2022. doi: 10.1038/s41467-022-31511-0.

[8] Sophia Lambert, Jakub Voznica, and Hélène Morlon. Deep learning from phylogenies for diversification analyses. *Systematic Biology*, 72(6):1262–1279, 2023. doi: 10.1093/sysbio/syad044.

[9] Tanja Stadler. Simulating trees with a fixed number of extant species. *Systematic Biology*, 60 (5):676–684, 2011. doi: 10.1093/sysbio/syr029.

[10] Sebastian Lequime, Paul Bastide, Simon Dellicour, Philippe Lemey, and Guy Baele. nosoi: A stochastic agent-based transmission chain simulation framework in R. *Methods in Ecology and Evolution*, 11(8):1002–1007, 2020. doi: 10.1111/2041-210X.13422.

[11] Franz Baumdicker, Gertjan Bisschop, Daniel Goldstein, Graham Gower, Aaron P. Ragsdale, Georgia Tsambos, Sha Zhu, Bjarki Eldon, E. Castedo Ellerman, Jared G. Galloway, Ariella L. Gladstein, Gregor Gorjanc, Bing Guo, Ben Jeffery, Warren W. Kretzschmar, Konrad Lohse, Michael Matschiner, Dominic Nelson, Nathaniel S. Pope, Consuelo D. Quinto-Cortés, Murillo F. Rodrigues, Kumar Saunack, Thibaut Sellinger, Kevin Thornton, Hugo van Kemenade, Anthony W. Wohns, Yan Wong, Simon Gravel, Andrew D. Kern, Jere Koskela, Peter L. Ralph, and Jerome Kelleher. Efficient ancestry and mutation simulation with msprime 1.0. *Genetics*, 220(3):iyab229, 2022. doi: 10.1093/genetics/iyab229.

[12] Joshua A. Justison, Claudia Solís-Lemus, and Tracy A. Heath. SiPhyNetwork: An R package for simulating phylogenetic networks. *Methods in Ecology and Evolution*, 14(7):1687–1698, 2023. doi: 10.1111/2041-210X.14116.

[13] Timothy G. Vaughan. ReMASTER: improved phylodynamic simulation for BEAST 2.7. *Bioinformatics*, 40(1):btae015, 2024. doi: 10.1093/bioinformatics/btae015.

[14] Daniel T. Gillespie. Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25):2340–2361, 1977. doi: 10.1021/j100540a008.

[15] Michael A. Gibson and Jehoshua Bruck. Efficient exact stochastic simulation of chemical systems with many species and many channels. *The Journal of Physical Chemistry A*, 104(9): 1876–1889, 2000. doi: 10.1021/jp993732q.

[16] David F. Anderson. A modified next reaction method for simulating chemical systems with time dependent propensities and delays. *The Journal of Chemical Physics*, 127(21):214107, 2007. doi: 10.1063/1.2799998.

[17] Tanja Stadler. Sampling-through-time in birth–death trees. *Journal of Theoretical Biology*, 267(3):396–404, 2010. doi: 10.1016/j.jtbi.2010.09.010.

[18] Tanja Stadler, Denise Kühnert, Sebastian Bonhoeffer, and Alexei J. Drummond. Birth–death skyline plot reveals temporal changes of epidemic spread in HIV and hepatitis C virus (HCV). *Proceedings of the National Academy of Sciences*, 110(1):228–233, 2013. doi: 10.1073/pnas.1207965110.

[19] Denise Kühnert, Tanja Stadler, Timothy G. Vaughan, and Alexei J. Drummond. Phylodynamics with migration: A computational framework to quantify population structure from genomic data. *Molecular Biology and Evolution*, 33(8):2102–2116, 2016. doi: 10.1093/molbev/msw064.

[20] Wayne P. Maddison, Peter E. Midford, and Sarah P. Otto. Estimating a binary character's effect on speciation and extinction. *Systematic Biology*, 56(5):701–710, 2007. doi: 10.1080/10635150701607033.

[21] Morihiro Notohara. The coalescent and the genealogical process in geographically structured population. *Journal of Mathematical Biology*, 29(1):59–75, 1990. doi: 10.1007/BF00173909.

[22] Alexei J. Drummond, Geoff K. Nicholls, Allen G. Rodrigo, and Wiremu Solomon. Estimating mutation parameters, population history and genealogy simultaneously from temporally spaced sequence data. *Genetics*, 161(3):1307–1320, 2002. doi: 10.1093/genetics/161.3.1307.

[23] J. F. C. Kingman. The coalescent. *Stochastic Processes and their Applications*, 13(3):235–248, 1982. doi: 10.1016/0304-4149(82)90011-4.

[24] R. C. Griffiths and Simon Tavaré. Sampling theory for neutral alleles in a varying environment. *Philosophical Transactions of the Royal Society of London. Series B*, 344(1310):403–410, 1994. doi: 10.1098/rstb.1994.0079.

[25] William Ogilvy Kermack and A. G. McKendrick. A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society of London. Series A*, 115(772):700–721, 1927. doi: 10.1098/rspa.1927.0118.

[26] Nhan Ly-Trong, Suha Naser-Khdour, Robert Lanfear, and Bui Quang Minh. AliSim: a fast and versatile phylogenetic sequence simulator for the genomic era. *Molecular Biology and Evolution*, 39(5):msac092, 2022. doi: 10.1093/molbev/msac092.

[27] Anna Zhukova and Olivier Gascuel. Accounting for contact tracing in epidemiological birth-death models. *PLOS Computational Biology*, 21(5):e1012461, 2025. doi: 10.1371/journal.pcbi.1012461.

[28] Theodore E. Harris. *The Theory of Branching Processes*. Springer, Berlin, 1963.